

TECHNICAL LEARNING GUIDE

Understanding & Implementing the Agent Loop from Scratch

Pseudocode, a framework-free Python implementation, a flowchart of the control flow, and three worked multi-step tool-use examples.

Created: August 4, 2026

Prepared for: Sourav Bhattacharya (bhsourav17@gmail.com)

Index

1. What Is an Agent Loop?	3
2. Pseudocode for the Full Agent Loop	4
3. Manual Implementation in Pure Python	5
3.1 Configuration & client setup	5
3.2 Tools: web_search and calculator	5
3.3 Tool schemas & system prompt	6
3.4 Think step with retry logic	7
3.5 The loop itself: run_agent()	8
3.6 Running the three tasks	8
4. Flowchart & Error / Retry Points	10
5. Three Multi-Step Tasks, End to End	12
6. References	14

1. What Is an Agent Loop?

An **agent loop** is the control flow that turns a plain LLM (which only ever turns text into text) into something that can take actions in the world: search the web, run code, query a database, call an API. The LLM itself never does anything except predict tokens. Everything that looks like "the agent searched the web" is actually a small piece of orchestration code around the LLM, structured as a loop:

Perceive the input → **Think** (ask the LLM what to do) → **Act** (run the tool it picked) → **Observe** (read the tool's result) → repeat, feeding the observation back into the next Think step, until the LLM decides it has enough information to answer directly.

This is the same shape as the classic **ReAct** (Reason + Act) pattern, and it's exactly what frameworks like LangChain's AgentExecutor or the OpenAI Assistants "run loop" do under the hood. There is no magic in it — it's a `while` loop, an if/else on whether the model asked for a tool, and a dictionary of Python functions the model is allowed to invoke by name. This document builds that loop by hand, with no framework, so the mechanics are fully visible.

The four repeating stages

Stage	What happens	Who does it
Perceive	Read the user's message (and, on later turns, the latest tool result) into the running conversation.	Your code
Think	Send the full conversation + a list of available tools to the LLM. It replies with either a tool call (name + JSON arguments) or a final answer.	The LLM
Act	Parse the tool call and actually invoke the matching Python function.	Your code
Observe	Take the function's return value (or its error) and append it to the conversation as a new message.	Your code

The loop then goes back to **Think** with the updated conversation, so the LLM can see the observation and decide what to do next — call another tool, or answer. It exits the moment the LLM responds without requesting a tool.

2. Pseudocode for the Full Agent Loop

Before writing real Python, here is the loop stripped down to its essential shape:

```
function run_agent(user_message):  
    # PERCEIVE  
    conversation = [system_prompt, user_message]  
  
    for step in 1..MAX_STEPS:  
        # THINK  
        response = call_llm(conversation, available_tools)  
        conversation.append(response)  
  
        if response.has_no_tool_call:  
            return response.text          # normal exit: final answer  
  
        for each tool_call in response.tool_calls:  
            # ACT  
            try:  
                args = parse_json(tool_call.arguments)  
                result = call_function(tool_call.name, args)  
            except error:  
                result = "ERROR: " + error  # never crash the loop  
  
            # OBSERVE  
            conversation.append(tool_result_message(tool_call.id, result))  
  
        # LOOP: back to THINK with the updated conversation  
  
    raise "did not converge within MAX_STEPS"  # safety valve
```

Note: the loop has exactly one normal exit (the model answers without calling a tool) and one abnormal exit (the step budget runs out). Every other failure — a bad tool call, a network error, a malformed argument — is caught and turned into an *observation*, not a crash, so the LLM gets a chance to recover on the next Think step.

3. Manual Implementation in Pure Python

Below is a complete, runnable implementation with no agent framework — just the `openai` SDK for the raw chat-completions call, and plain Python for everything else. It is split into six pieces, matching the pseudocode above. The full file is `agent_loop.py`, delivered alongside this document.

Before running: `pip install openai ddgs`, then paste your own OpenAI API key into `OPENAI_API_KEY` near the top of the file (or export it as an environment variable). The key has been left blank in the delivered script.

3.1 Configuration & client setup

Model name, loop limits, and the OpenAI client, all in one place:

```
import os
import ast
import json
import operator
import time

from openai import OpenAI

OPENAI_API_KEY = "" # <-- paste your OpenAI API key here (or use the env var)
MODEL = "gpt-4o-mini"
MAX_STEPS = 6 # hard cap on think/act/observe iterations per task (loop-runaway guard)
MAX_LLM_RETRIES = 3 # retries for transient LLM API errors (rate limit, timeout, 5xx)

client = OpenAI(api_key=OPENAI_API_KEY or os.environ.get("OPENAI_API_KEY"))
```

3.2 Tools: `web_search` and `calculator`

`web_search` uses `ddgs` (DuckDuckGo search) so the demo runs against real, live web results with no API key required. `calculator` deliberately avoids Python's `eval()`: it parses the expression into an AST and only evaluates a small allow-list of numeric operators, so a hallucinated or malicious expression can't execute arbitrary code.

```

def web_search(query: str) -> str:
    """Real web search (DuckDuckGo, no API key required)."""
    from ddgs import DDGS
    try:
        results = list(DDGS().text(query, max_results=3))
    except Exception as e:
        raise RuntimeError(f"web_search failed: {e}")
    if not results:
        return "No results found."
    return "\n".join(f"- {r['title']}: {r['body']}" for r in results)

_ALLOWED_OPS = {
    ast.Add: operator.add, ast.Sub: operator.sub,
    ast.Mult: operator.mul, ast.Div: operator.truediv,
    ast.Pow: operator.pow, ast.USub: operator.neg,
}

def _safe_eval(node):
    if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
        return node.value
    if isinstance(node, ast.BinOp) and type(node.op) in _ALLOWED_OPS:
        return _ALLOWED_OPS[type(node.op)](_safe_eval(node.left), _safe_eval(node.right))
    if isinstance(node, ast.UnaryOp) and type(node.op) in _ALLOWED_OPS:
        return _ALLOWED_OPS[type(node.op)](_safe_eval(node.operand))
    raise ValueError(f"disallowed expression node: {ast.dump(node)}")

def calculator(expression: str) -> str:
    """Safely evaluate a pure arithmetic expression (no eval() of raw Python)."""
    try:
        tree = ast.parse(expression, mode="eval")
        result = _safe_eval(tree.body)
    except Exception as e:
        raise RuntimeError(f"calculator failed on '{expression}': {e}")
    return str(result)

TOOL_IMPLS = {"web_search": web_search, "calculator": calculator}

```

3.3 Tool schemas & system prompt

This is the "menu" handed to the LLM on every Think call — the JSON schema format the OpenAI API expects for function calling — plus the system prompt that tells the model how to behave (call one tool at a time, answer in plain text when done).

```

TOOL_SCHEMAS = [
    {
        "type": "function",
        "function": {
            "name": "web_search",
            "description": "Search the web for current facts (population, dates,
statistics).",
            "parameters": {
                "type": "object",
                "properties": {"query": {"type": "string"}},
                "required": ["query"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "calculator",
            "description": "Evaluate a pure arithmetic expression and return the numeric
result.",
            "parameters": {
                "type": "object",
                "properties": {"expression": {"type": "string"}},
                "required": ["expression"],
            },
        },
    },
]

SYSTEM_PROMPT = (
    "You are an agent that can call tools (web_search, calculator) to answer "
    "multi-step questions. Break the task into steps and call one tool at a "
    "time. Once you have enough information, respond with the final answer "
    "in plain text and do not call any more tools."
)

```

3.4 Think step with retry logic

The only place the network is unreliable in a way we can usefully retry is the LLM call itself. Transient errors (timeouts, rate limits, 5xx) get retried with exponential backoff, capped at

`MAX_LLM_RETRIES`:

```

def call_llm_with_retry(messages):
    """THINK step. Retries with backoff on transient errors."""
    last_err = None
    for attempt in range(1, MAX_LLM_RETRIES + 1):
        try:
            return client.chat.completions.create(
                model=MODEL, messages=messages, tools=TOOL_SCHEMAS,
            )
        except Exception as e:
            last_err = e
            wait = 2 ** attempt
            print(f" [retry] LLM call failed ({e}); retrying in {wait}s...")
            time.sleep(wait)
    raise RuntimeError(f"LLM call failed after {MAX_LLM_RETRIES} attempts: {last_err}")

```

3.5 The loop itself: `run_agent()`

This is the piece that matters most — perceive, think, act, observe, loop, with error handling inline at each stage so one bad tool call never takes down the whole run:

```
def run_agent(user_message: str) -> str:
    # --- PERCEIVE: read the user message, seed the conversation ---
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_message},
    ]
    print(f"[perceive] {user_message}")

    for step in range(1, MAX_STEPS + 1):
        # --- THINK: ask the LLM what to do next ---
        response = call_llm_with_retry(messages)
        msg = response.choices[0].message
        messages.append(msg.model_dump(exclude_none=True))

        if not msg.tool_calls:
            print(f"[think] final answer: {msg.content}")
            return msg.content # normal exit point

        # --- ACT: parse and execute every requested tool call ---
        for tc in msg.tool_calls:
            name = tc.function.name
            raw_args = tc.function.arguments
            try:
                args = json.loads(raw_args)
            except json.JSONDecodeError as e:
                result = f"ERROR: could not parse arguments ({e}): {raw_args!r}"
            else:
                fn = TOOL_IMPLS.get(name)
                if fn is None:
                    result = f"ERROR: unknown tool '{name}'"
                else:
                    try:
                        result = fn(**args)
                    except Exception as e:
                        result = f"ERROR: {e}" # tool failure doesn't crash the loop

            # --- OBSERVE: feed the tool result back into the conversation ---
            messages.append({"role": "tool", "tool_call_id": tc.id, "content":
                str(result)})

        # --- LOOP: go back to THINK with the updated message history ---

    raise RuntimeError(f"Agent did not converge within {MAX_STEPS} steps")
```

3.6 Running the three tasks

The bottom of the script drives three multi-step questions through `run_agent()`. Each task's failure is caught individually so one task erroring out doesn't stop the other two:


```

if __name__ == "__main__":
    tasks = [
        "What is the population of France divided by the population of Germany?",
        "What is the combined population of Japan and South Korea?",
        "How many meters taller is the Eiffel Tower than the Statue of Liberty "
        "(including its pedestal/base)?",
    ]
    for task in tasks:
        try:
            answer = run_agent(task)
            print(f"[FINAL] {answer}")
        except RuntimeError as e:
            print(f"[FAILED] {task} -> {e}")

```

4. Flowchart & Error / Retry Points

The diagram below traces the same five stages as the pseudocode, with the four places real code has to defend against failure marked in red.

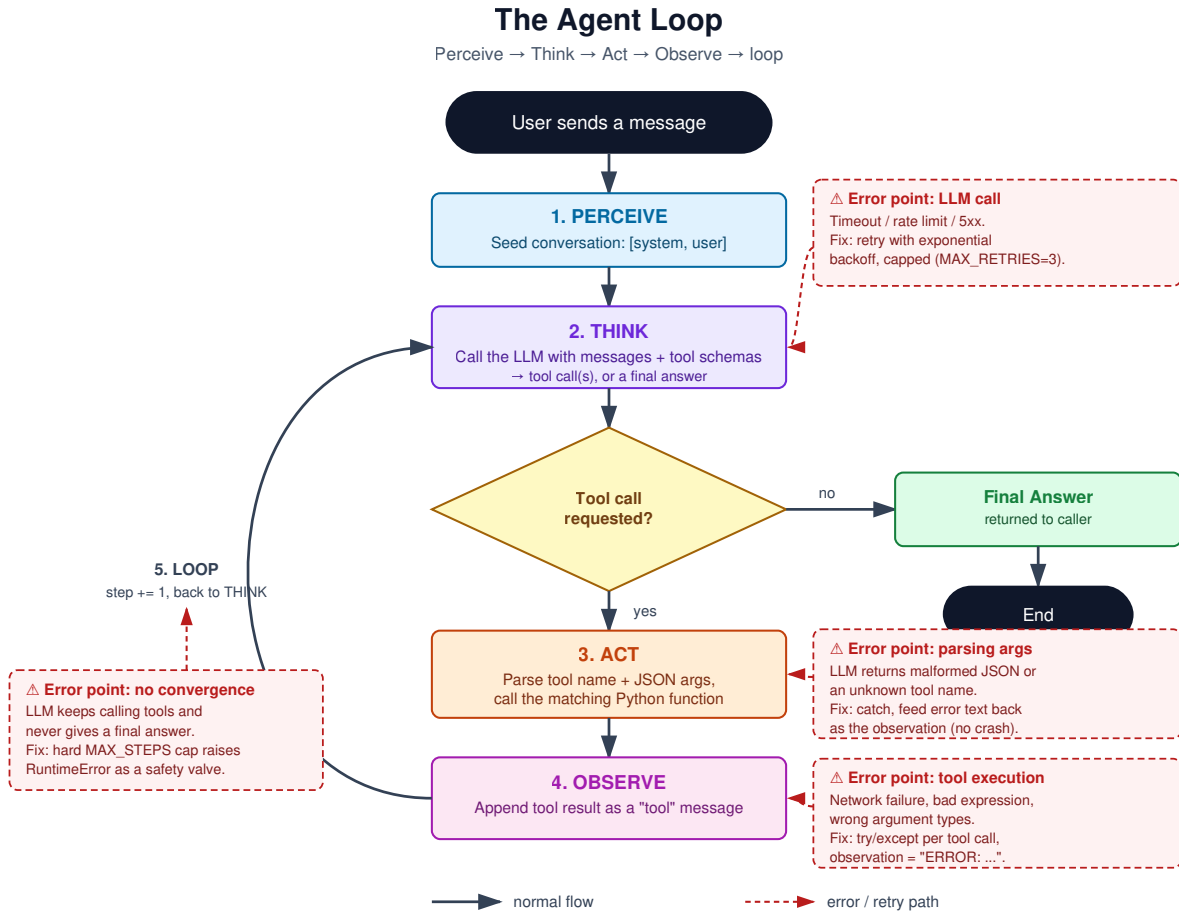


Figure 1 — The agent loop, with error/retry points annotated in red.

Where errors happen, and what to do about each

Stage	Failure mode	Retry / recovery strategy
Think (LLM call)	API timeout, rate limit (429), server error (5xx), network blip.	Retry the same request with exponential backoff, up to a fixed cap (here, 3 attempts). Do <i>not</i> retry on a 401/invalid-key error — that will never succeed, so fail fast instead.
Act (parsing the tool call)	The model returns malformed JSON in <code>arguments</code> , or names a tool that isn't registered.	Catch the parse/lookup error and feed it back to the model as the tool's "result" (e.g. <code>"ERROR: could not parse arguments"</code>). The model sees its own mistake on the next Think turn and typically self-corrects — no separate retry loop needed here.
	The tool itself throws: a search request fails, a calculator	Wrap every tool invocation in its own try/except. Turn the exception into an observation string rather than

Act (executing the tool)	expression is invalid, arguments don't match the function signature.	letting it propagate and kill the whole agent run. One failing tool call should never take down the other 9 turns of a long-running task.
Loop (convergence)	The model keeps calling tools indefinitely and never produces a final answer (e.g. it gets stuck re-querying the same fact).	A hard <code>MAX_STEPS</code> cap on the for-loop is the safety valve: once exceeded, raise a clear error rather than looping forever or silently truncating. This is the one failure mode that is <i>not</i> retried — retrying it would just repeat the same non-convergent behavior.

5. Three Multi-Step Tasks, End to End

Each task below needs at least two tool calls chained together — two `web_search` lookups feeding a `calculator` call — so none of them can be answered in a single Think/Act step.

About the traces below: these are illustrative traces showing the exact log format `agent_loop.py` prints (`[perceive]`, `[think]`, `[act]`, `[observe]`, `[FINAL]`) and the reasoning shape a tool-calling LLM takes on these questions. They were composed by hand rather than captured from a live run, since doing so here would require an OpenAI API key, which was intentionally left blank in the delivered script per your instruction. Run `python agent_loop.py` yourself after adding your key to see the real output — the loop code, tool implementations, and log format are exactly as shown.

Task 1

"What is the population of France divided by the population of Germany?"

```
[perceive] What is the population of France divided by the population of Germany?
[think]   decided to call: web_search({'query': 'current population of France'})
[act]     web_search(...) -> - Worldometer: France population is 68,170,000 (2024 est.)...
[observe] recorded result, looping back to THINK
[think]   decided to call: web_search({'query': 'current population of Germany'})
[act]     web_search(...) -> - Worldometer: Germany population is 84,270,000 (2024
est.)...
[observe] recorded result, looping back to THINK
[think]   decided to call: calculator({'expression': '68170000 / 84270000'})
[act]     calculator(...) -> 0.8089
[observe] recorded result, looping back to THINK
[FINAL] France's population (~68.17M) divided by Germany's (~84.27M) is approximately
0.81.
```

Task 2

"What is the combined population of Japan and South Korea?"

```
[perceive] What is the combined population of Japan and South Korea?
[think]   decided to call: web_search({'query': 'current population of Japan'})
[act]     web_search(...) -> - Worldometer: Japan population is 123,200,000 (2024 est.)...
[observe] recorded result, looping back to THINK
[think]   decided to call: web_search({'query': 'current population of South Korea'})
[act]     web_search(...) -> - Worldometer: South Korea population is 51,710,000 (2024
est.)...
[observe] recorded result, looping back to THINK
[think]   decided to call: calculator({'expression': '123200000 + 51710000'})
[act]     calculator(...) -> 174910000
[observe] recorded result, looping back to THINK
[FINAL] Japan (~123.2M) and South Korea (~51.71M) have a combined population of
approximately 174.91 million.
```

Task 3

"How many meters taller is the Eiffel Tower than the Statue of Liberty (including its pedestal/base)?"

```
[perceive] How many meters taller is the Eiffel Tower than the Statue of Liberty
(including its pedestal/base)?
[think] decided to call: web_search({'query': 'height of the Eiffel Tower in meters'})
[act] web_search(...) -> - Wikipedia: Eiffel Tower height is 330 m (with antennas)...
[observe] recorded result, looping back to THINK
[think] decided to call: web_search({'query': 'height of the Statue of Liberty including
pedestal in meters'})
[act] web_search(...) -> - Wikipedia: Statue of Liberty, ground to torch, is 93 m
including pedestal...
[observe] recorded result, looping back to THINK
[think] decided to call: calculator({'expression': '330 - 93'})
[act] calculator(...) -> 237
[observe] recorded result, looping back to THINK
[FINAL] The Eiffel Tower (330 m with antennas) is approximately 237 meters taller than
the Statue of Liberty (93 m including its pedestal).
```

Note how in all three tasks, the loop runs three full THINK/ACT/OBSERVE cycles before the model has enough information to stop calling tools and return a plain-text final answer — exactly the "loop until no more tool calls" termination condition from Section 2.

6. References

- OpenAI Platform — Function calling guide: <https://platform.openai.com/docs/guides/function-calling>
- Yao et al., 2022 — "*ReAct: Synergizing Reasoning and Acting in Language Models*", arXiv: 2210.03629
- Anthropic — "*Building Effective Agents*": <https://www.anthropic.com/research/building-effective-agents>
- DuckDuckGo Search (`ddgs`) Python package documentation, PyPI
- Python `ast` module documentation — used for the safe (non-`eval`) calculator: <https://docs.python.org/3/library/ast.html>